AWS VPC Peering: Student Project Report

Oct 21, 2026

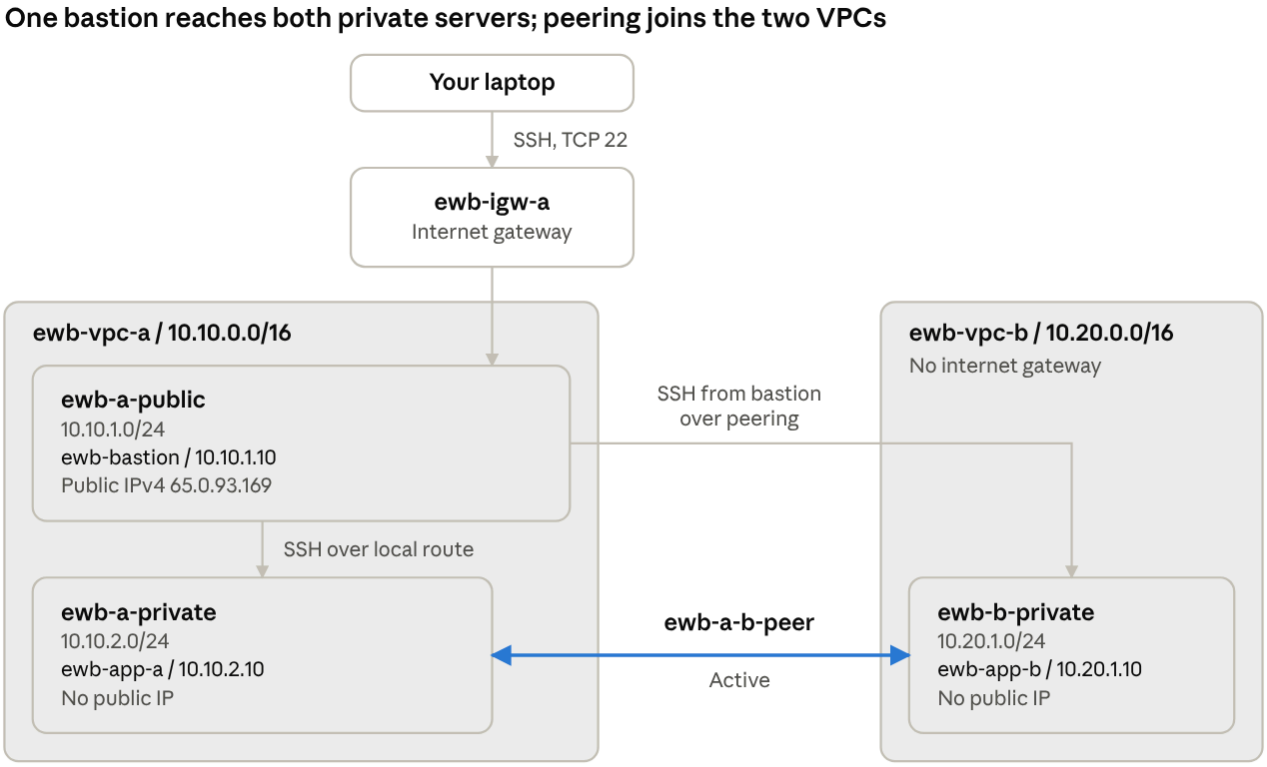
Two private VPCs connected with VPC peering, with a bastion host for administration and test evidence of private connectivity.

Project details

Field	Value
Student name	Jiya Mary Jacob
Batch	AWS Internship Batch 1
Project title	AWS VPC Peering
Region	Asia Pacific (Mumbai) / ap-south-1
Availability Zone	ap-south-1a
Start date	28 September 2026
Completion date	21 October 2026
Tags on every resource	Project = EWB-VPC-Peering, Student = Jiya

Architecture

One public bastion in VPC A reaches two private servers, and VPC A and VPC B are joined by one peering connection.



Highlighted line: the peering connection. Neither private server has a public IP.

architecture · 2 VPCs, 3 instances, 1 peering connection

Administration path: laptop to bastion to App-A or App-B using SSH ProxyJump. Project traffic: App-A and App-B over private IPs through the peering connection. VPC B has no internet gateway.

Resource inventory

All resources are tagged Project = EWB-VPC-Peering and Student = Jiya. VPC B has no internet gateway.

VPCs, subnets and internet gateway

Resource (Name tag)	ID	CIDR / detail
ewb-vpc-a	vpc-074f44afffd33e8b0	10.10.0.0/16
ewb-vpc-b	vpc-071028f353ae50f21	10.20.0.0/16
ewb-a-public	subnet-00a5804e737fb0429	10.10.1.0/24, ap-south-1a
ewb-a-private	subnet-0234c35664492b902	10.10.2.0/24, ap-south-1a
ewb-b-private	subnet-035b918b997acd8f7	10.20.1.0/24, ap-south-1a

Resource (Name tag)	ID	CIDR / detail
ewb-igw-a	igw-07c4520711c1268fa	Attached to ewb-vpc-a

Instances

All three run Amazon Linux 2023 on t3.micro.

Name	Instance ID	VPC / subnet	Private IP	Public IPv4	Security group
ewb-bastion	i-007009e53589bd038	ewb-vpc-a / ewb-a-public	10.10.1.10	65.0.93.169	ewb-sg-bastion
ewb-app-a	i-06031e6f18aa79a9f	ewb-vpc-a / ewb-a-private	10.10.2.10	None	ewb-sg-app-a
ewb-app-b	i-0c07959a5eb16b3be	ewb-vpc-b / ewb-b-private	10.20.1.10	None	ewb-sg-app-b

Security groups and root volumes

Resource	Security group ID	Root volume ID
ewb-bastion	sg-012fdb869c6b1077f (ewb-sg-bastion)	vol-0afcdea04bad7d627
ewb-app-a	sg-0a28c745319e85766 (ewb-sg-app-a)	vol-03efac937c3c63e15
ewb-app-b	sg-002f9b9e3f8015ec8 (ewb-sg-app-b)	vol-0426b221ab7fd62dd

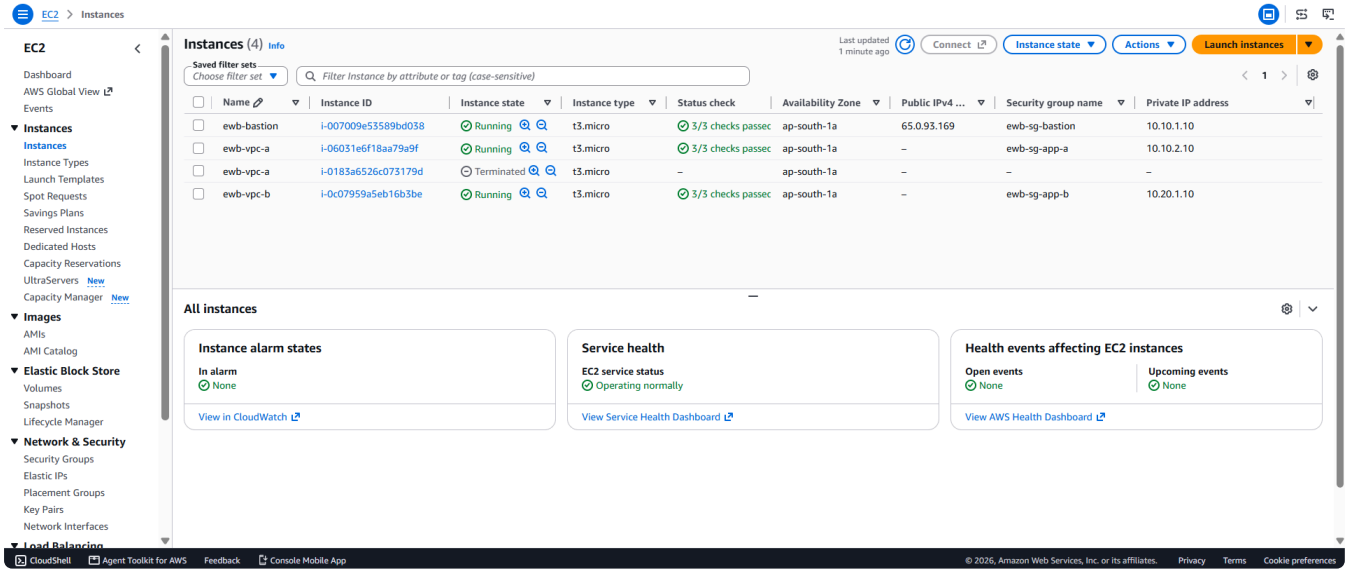


Figure 1. EC2 instance list: ewb-bastion, ewb-app-a and ewb-app-b running with 3/3 status checks passed, private IPs and the bastion public IPv4 (Checkpoint 02).

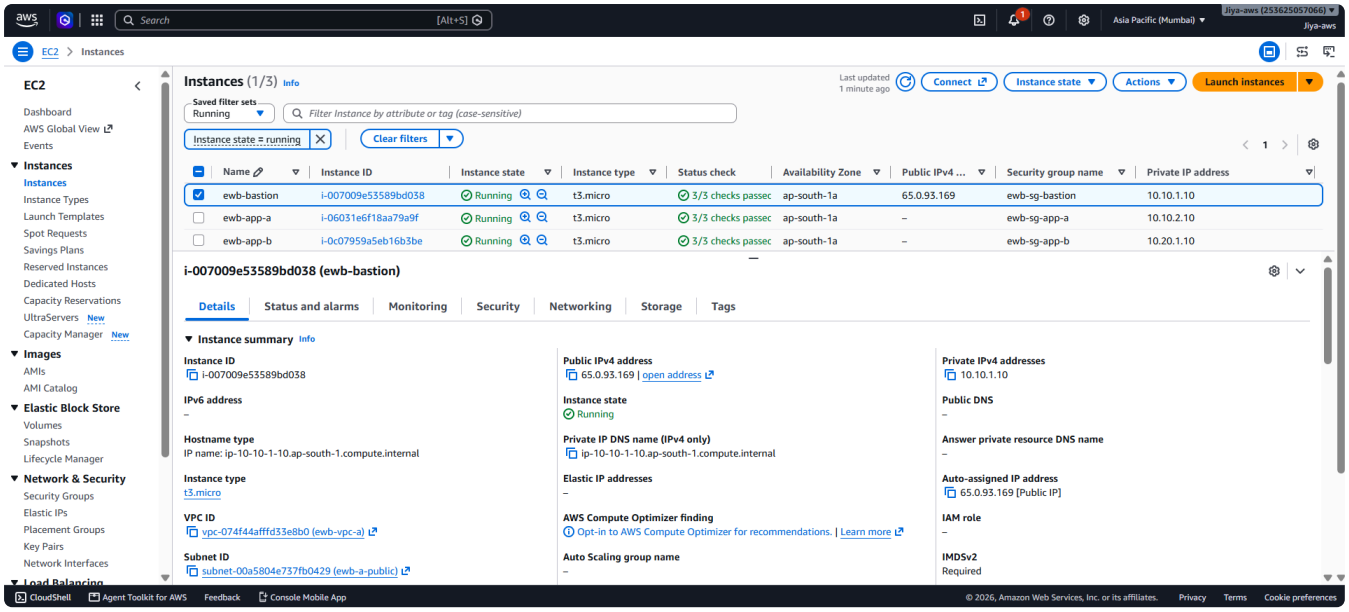


Figure 2. ewb-bastion instance details: private IP 10.10.1.10, public IPv4 65.0.93.169, in ewb-vpc-a / ewb-a-public.

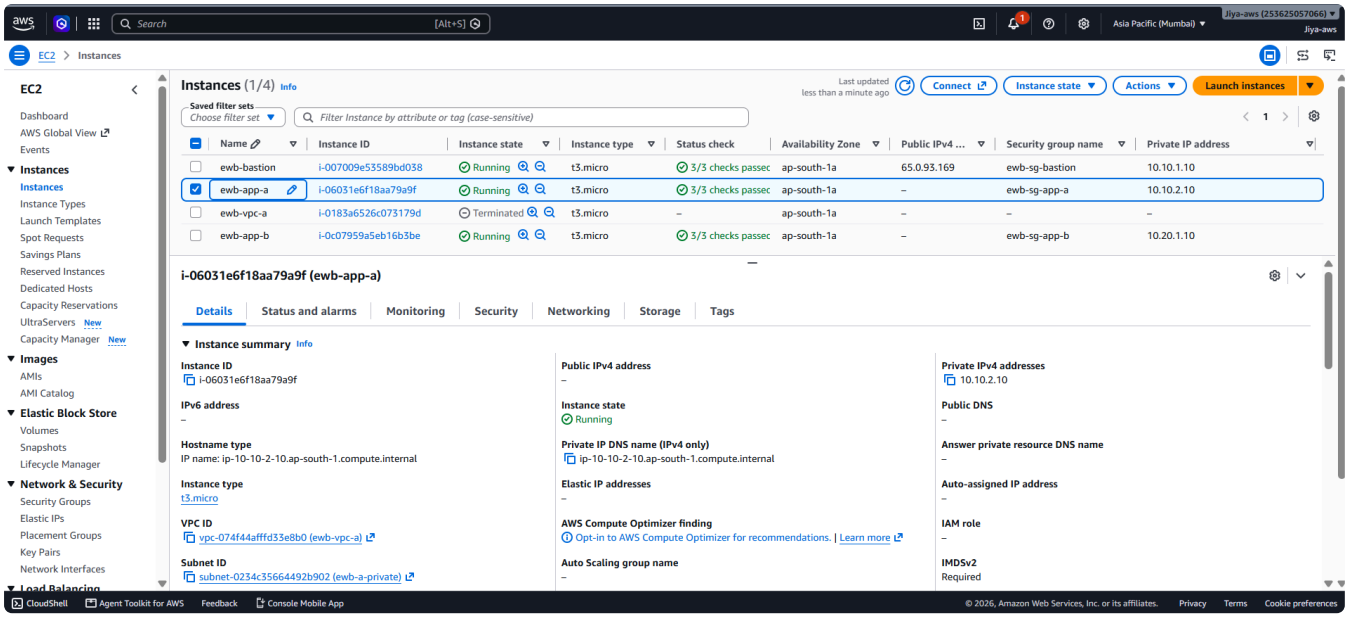


Figure 3. ewb-app-a instance details: private IP 10.10.2.10, in ewb-vpc-a / ewb-a-private.

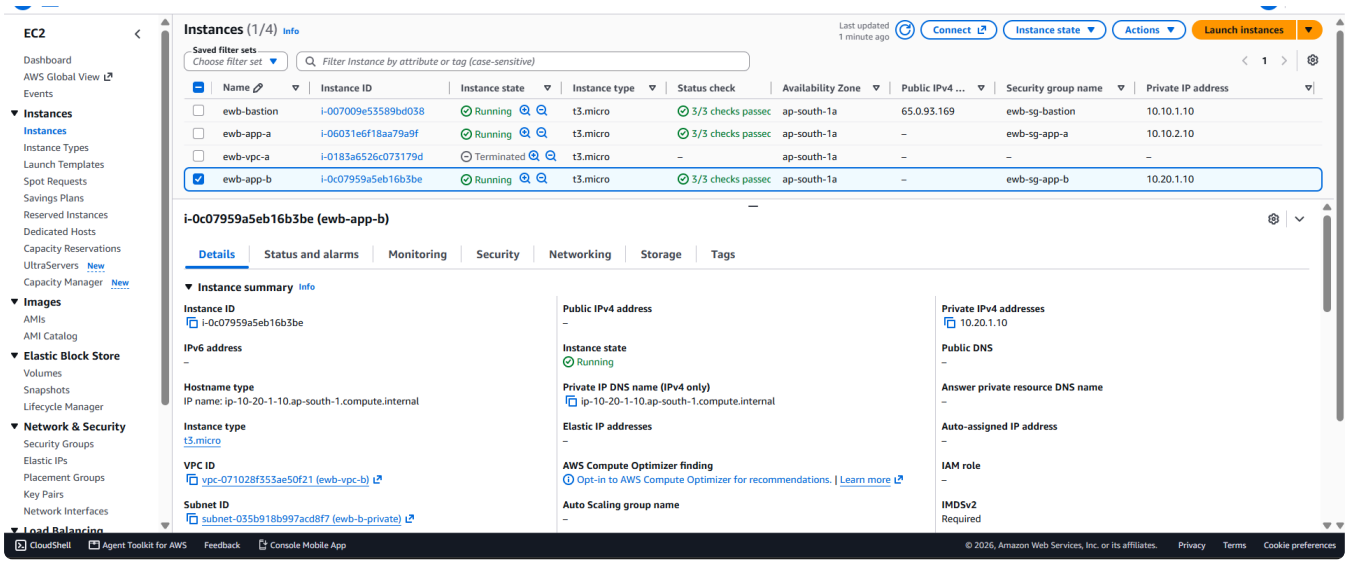


Figure 4. ewb-app-b instance details: private IP 10.20.1.10, in ewb-vpc-b / ewb-b-private.

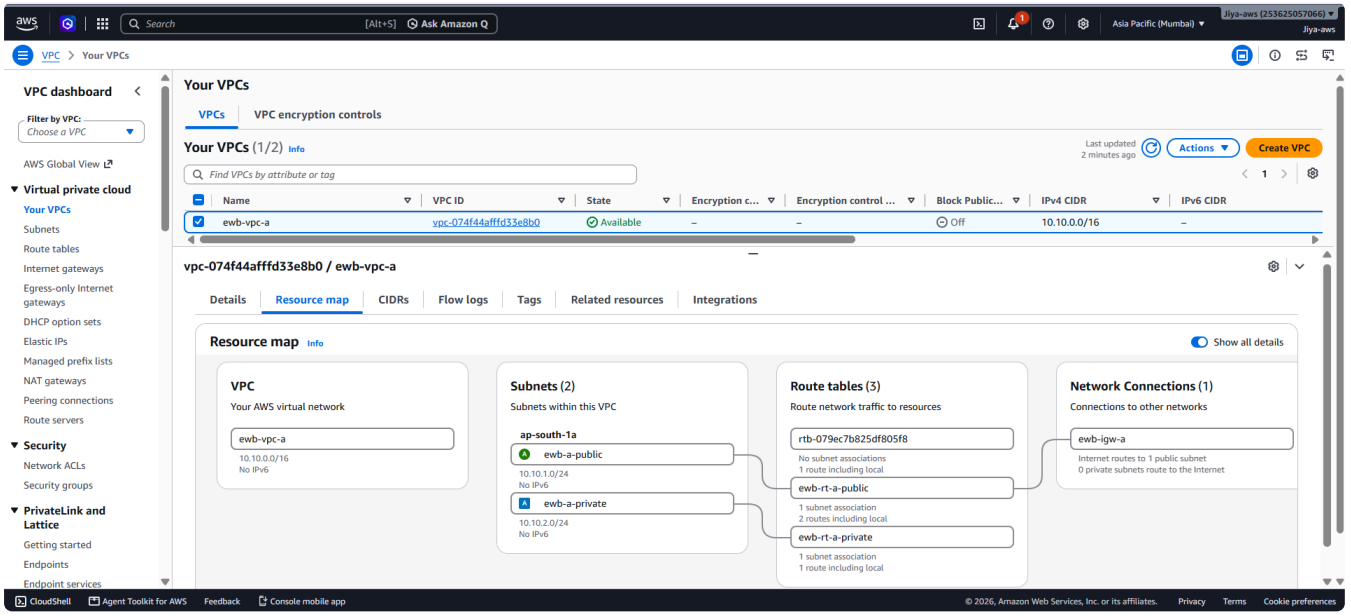


Figure 5. VPC resource map for ewb-vpc-a: two subnets, three route tables and the internet gateway.

Network build: route tables

Each subnet is explicitly associated with its own route table. Peering routes were added only after the peering connection became Active.

Route table	ID	Associated subnet	Routes
ewb-rt-a-public	rtb-0211d8a5965a3eda4	ewb-a-public	10.10.0.0/16 local; 0.0.0.0/0 igw-07c4520711c1268fa; 10.20.0.0/16 pcx-018895e855a398629

Route table	ID	Associated subnet	Routes
ewb-rt-a-private	rtb-0af2a2caab36a0585	ewb-a-private	10.10.0.0/16 local; 10.20.0.0/16 pcx-018895e855a398629
ewb-rt-b-private	rtb-04039a2202e5eb420	ewb-b-private	10.20.0.0/16 local; 10.10.0.0/16 pcx-018895e855a398629

The main route tables of both VPCs (rtb-079ec7b825df805f8 and rtb-0549dcd7b622b66b9) were left unchanged and have no explicit subnet associations.

The public subnet needs the 10.20.0.0/16 peering route because ProxyJump makes the bastion open its own connection to App-B. The more specific peering route is chosen over the internet default route.

ewb-rt-a-public

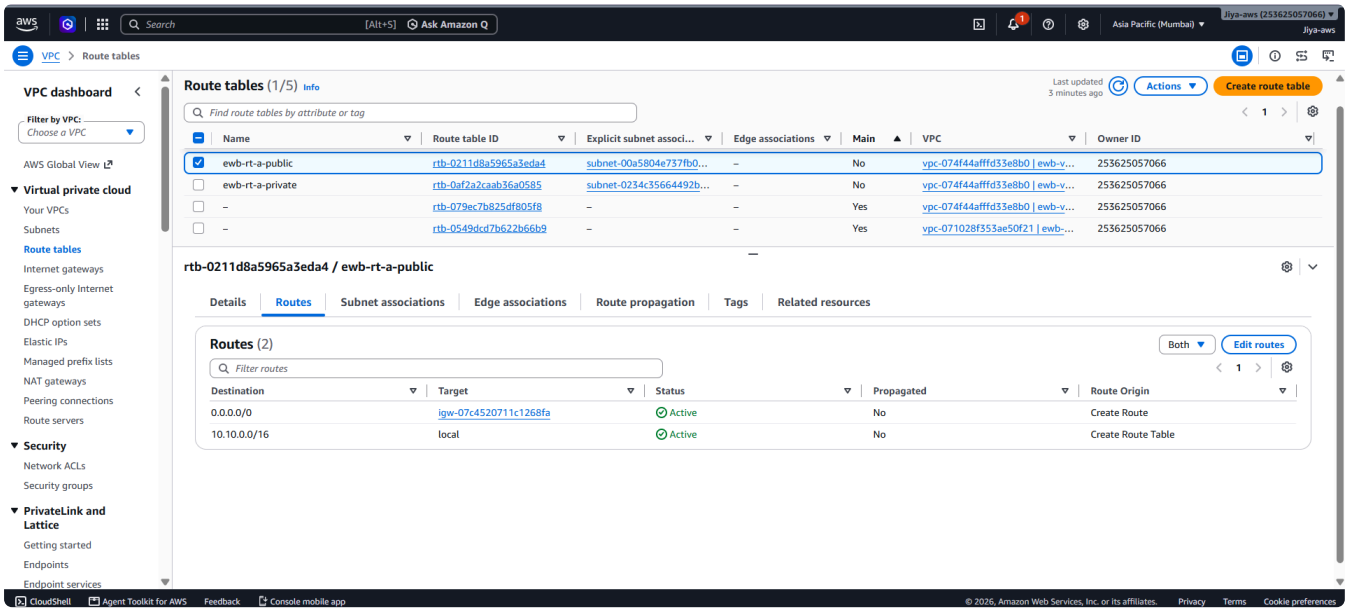


Figure 6. ewb-rt-a-public, Routes tab before peering: local route and 0.0.0.0/0 to the internet gateway.

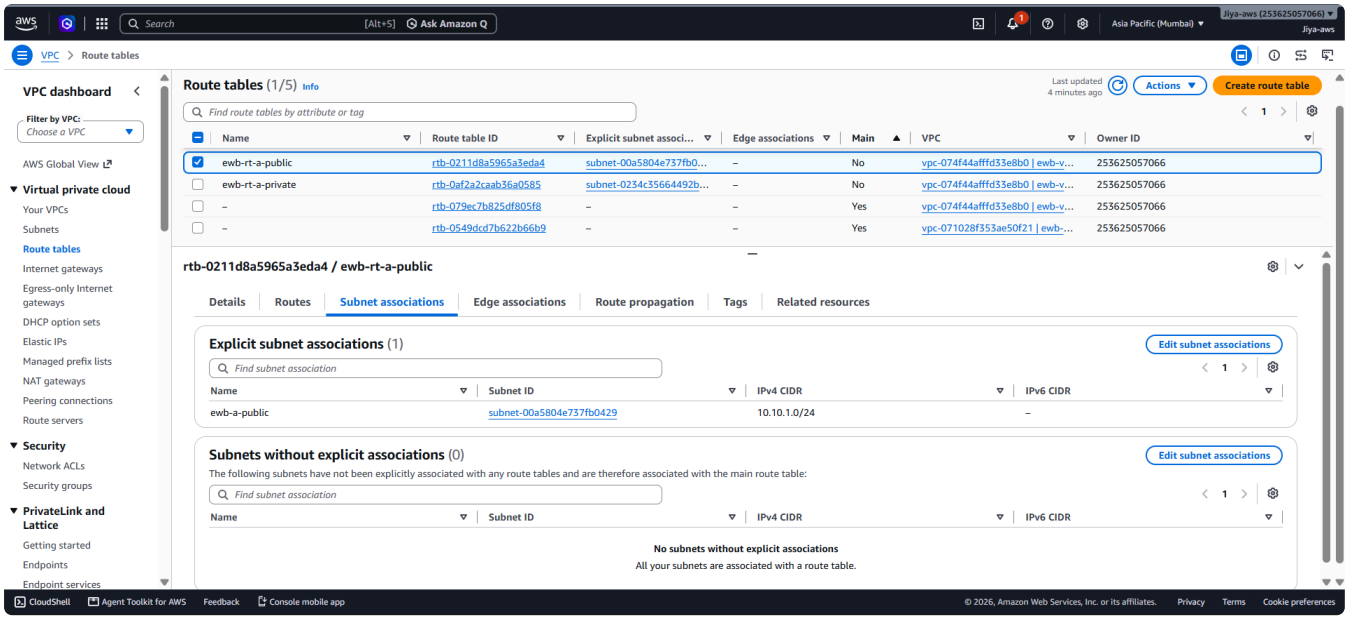


Figure 7. ewb-rt-a-public, Subnet associations tab before peering: ewb-a-public associated explicitly (Checkpoint 01).

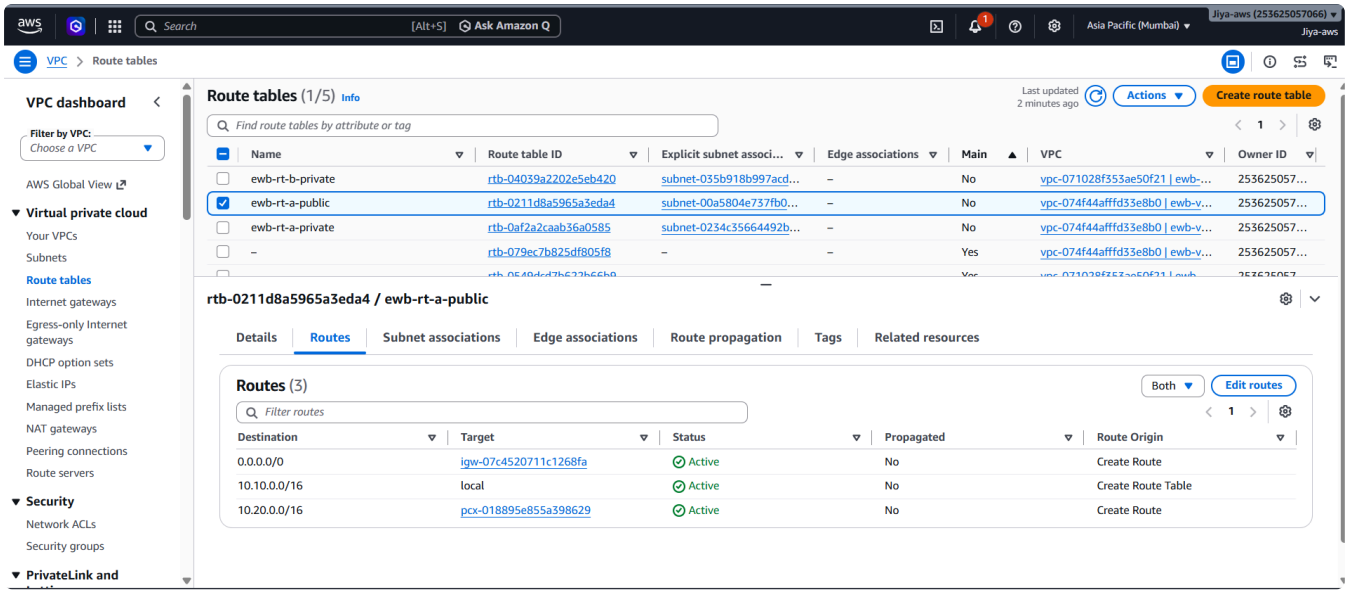


Figure 8. ewb-rt-a-public, Routes tab after peering: the 10.20.0.0/16 route to pcx-018895e855a398629 added (Checkpoint 04).

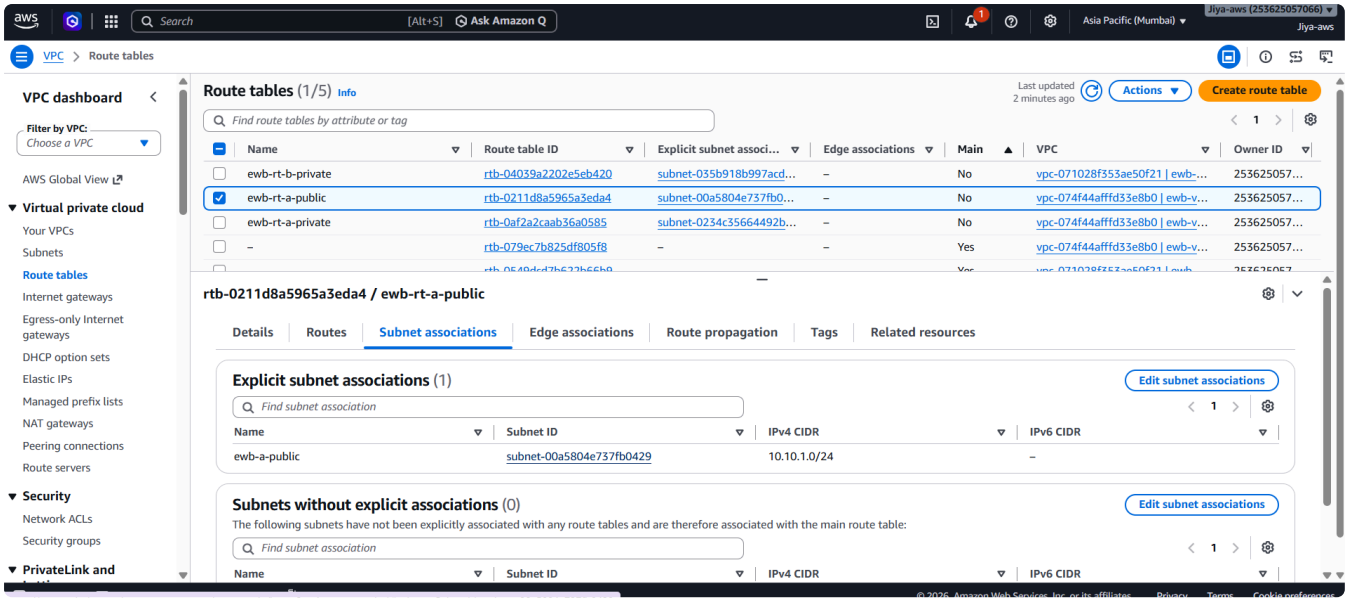


Figure 9. ewb-rt-a-public, Subnet associations tab after peering.

ewb-rt-a-private

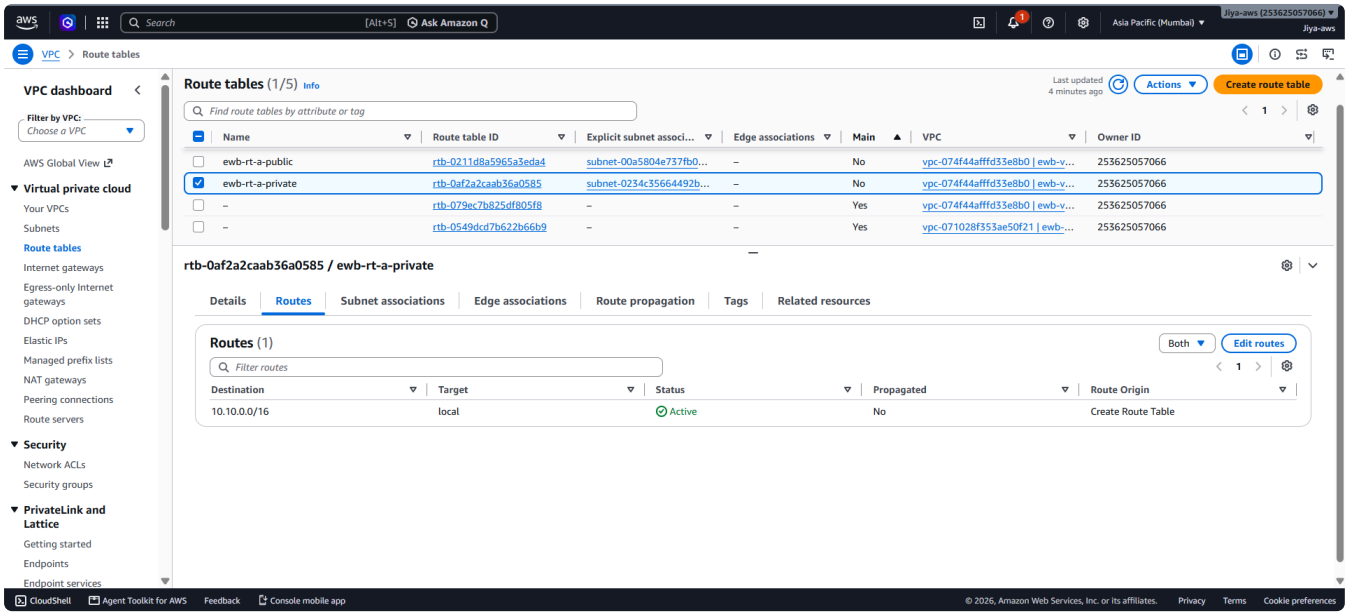


Figure 10. ewb-rt-a-private, Routes tab before peering: local route only (Checkpoint 01).

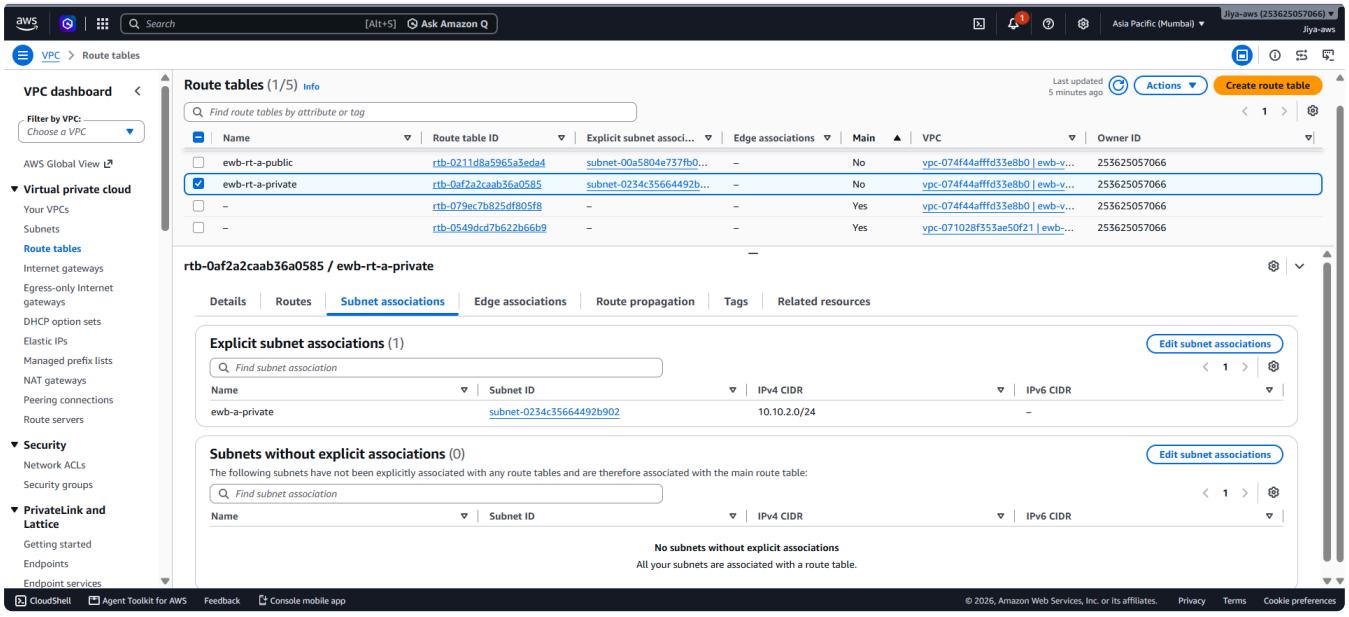


Figure 11. ewb-rt-a-private, Subnet associations tab before peering: ewb-a-private associated explicitly (Checkpoint 01).

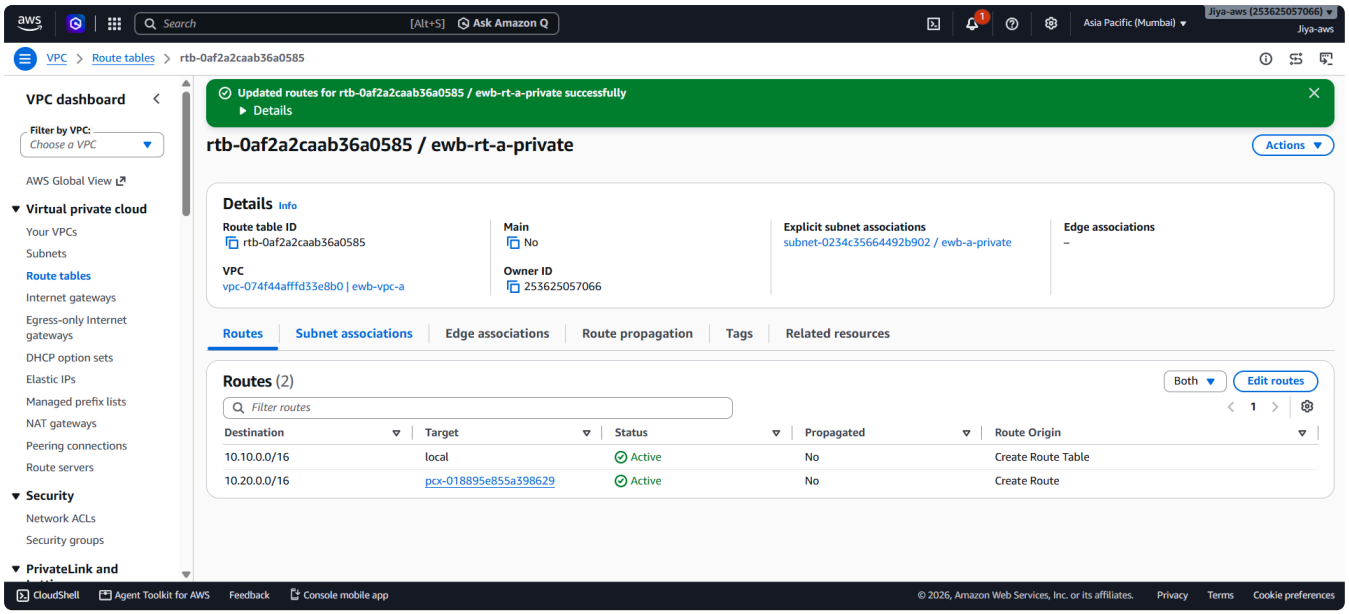


Figure 12. ewb-rt-a-private, Routes tab after peering: route update confirmed, 10.20.0.0/16 to pcx-018895e855a398629 (Checkpoint 04).

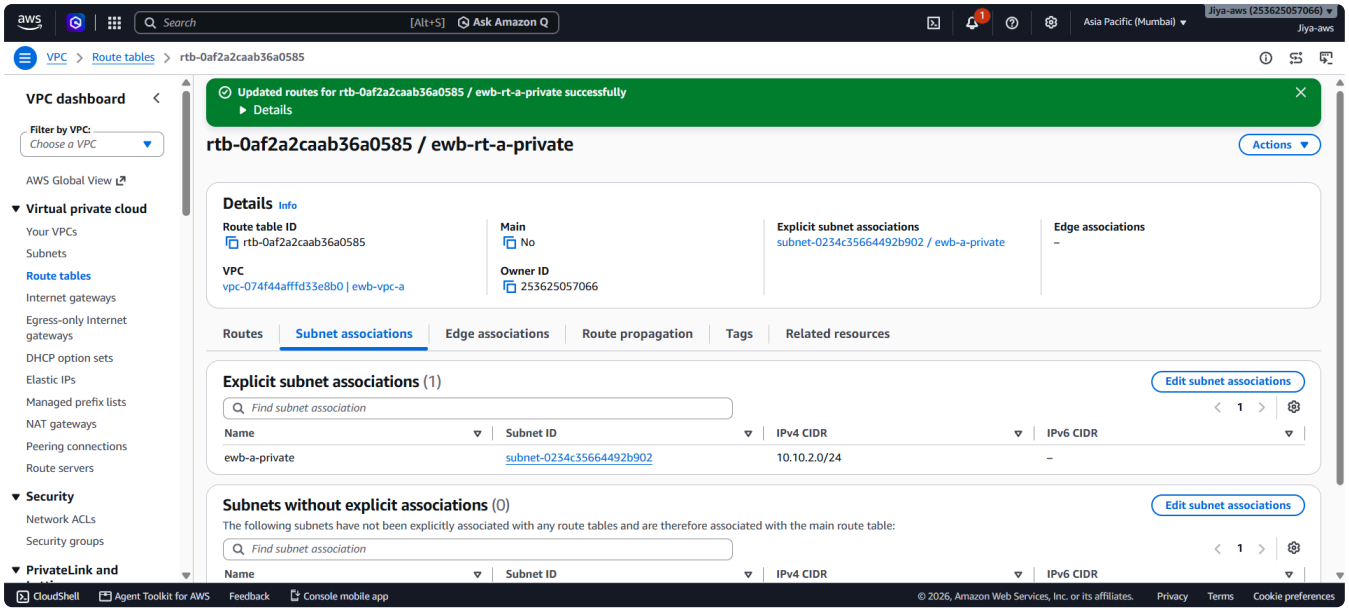


Figure 13. ewb-rt-a-private, Subnet associations tab after the route update.

ewb-rt-b-private

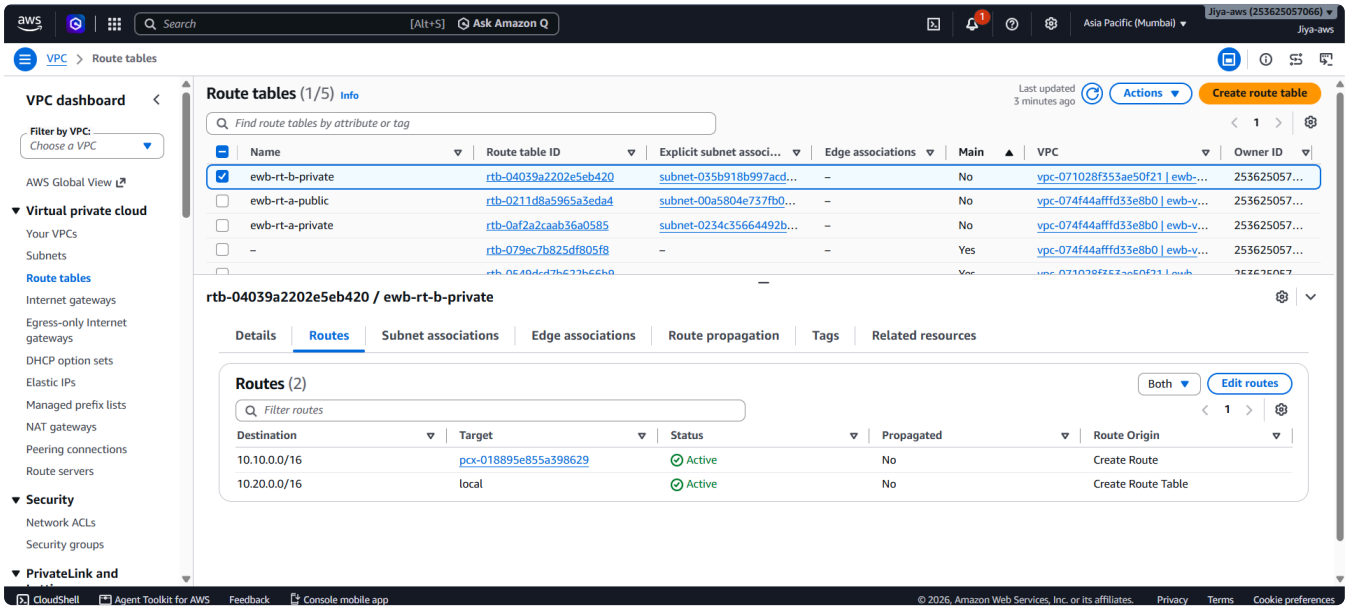


Figure 14. ewb-rt-b-private, Routes tab: local route and the return route 10.10.0.0/16 to pcx-018895e855a398629.

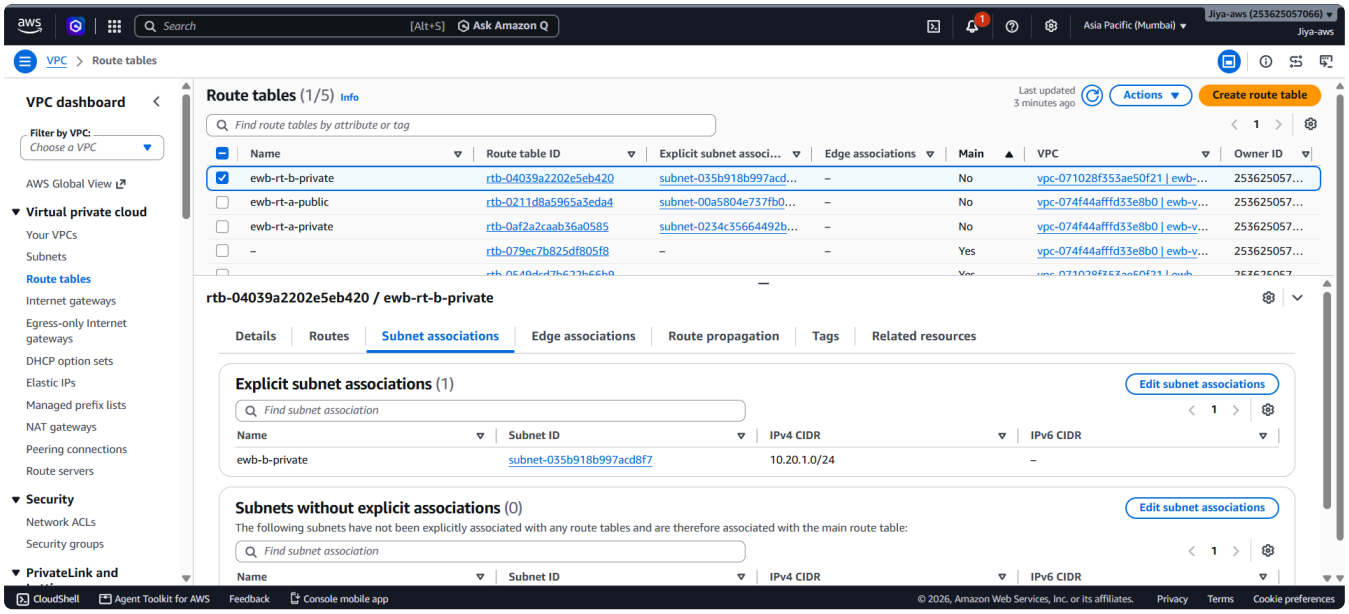


Figure 15. ewb-rt-b-private, Subnet associations tab: ewb-b-private associated explicitly.

Security groups

SSH and ping are allowed only from the specific lab hosts. Nothing is open to 0.0.0.0/0. Outbound rules keep the default allow-all.

Security group	Type	Port / protocol	Source
ewb-sg-bastion	SSH	TCP 22	65.0.93.169/32 (my public IP)
ewb-sg-app-a	SSH	TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-a	Custom ICMP, IPv4	Echo request	10.20.1.10/32 (App-B)
ewb-sg-app-b	SSH	TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-b	Custom ICMP, IPv4	Echo request	10.10.2.10/32 (App-A)

VPC peering connection

The request was accepted manually, which is required even when both VPCs are in the same account.

Field	Value
Name	ewb-a-b-peer
Peering connection ID	pcx-018895e855a398629

Field	Value
Requester VPC	ewb-vpc-a / 10.10.0.0/16
Accepter VPC	ewb-vpc-b / 10.20.0.0/16
Account / Region	My account / This Region
Status	Active

> **Screenshot to insert:** peering connection page showing name, Active status and both VPC IDs (Checkpoint 03).

Test evidence

All five tests passed. Hostnames recorded with `hostname` and `hostname -I`: App-A is ip-10-10-2-10.ap-south-1.compute.internal (10.10.2.10) and App-B is ip-10-20-1-10.ap-south-1.compute.internal (10.20.1.10).

Test	Host and command	Result
01 Baseline	App-A, before peering: `ping -c 4 10.20.1.10`	No replies, 100% packet loss
02 Forward path	App-A, after routes: `ping -c 4 10.20.1.10`	4 transmitted, 4 received, 0% packet loss
03 Reverse path	App-B, after routes: `ping -c 4 10.10.2.10`	4 transmitted, 4 received, 0% packet loss
04 Private SSH	Laptop: `ssh ewb-app-b`	Shell opened on App-B through the bastion
05 Isolation	EC2 details of App-A and App-B	Public IPv4 address empty on both

```
PS C:\Users\jiyam> ssh ewb-app-a
The authenticity of host '10.10.2.10 (<no hostip for proxy command>)' can't be established.
ED25519 key fingerprint is SHA256:Kr8rcKfoYPPs5OinYJMd2fx27e1UB4yfquQTUftjzF0.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.10.2.10' (ED25519) to the list of known hosts.
       #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-10-2-10 ~]$ hostname
ip-10-10-2-10.ap-south-1.compute.internal
[ec2-user@ip-10-10-2-10 ~]$ hostname-I
-bash: hostname-I: command not found
[ec2-user@ip-10-10-2-10 ~]$ hostname -I
10.10.2.10
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3107ms

[ec2-user@ip-10-10-2-10 ~]$
```

Figure 16. Test 01, App-A (ip-10-10-2-10), before peering: ping -c 4 10.20.1.10 gets 0 replies, 100% packet loss. hostname and hostname -I shown.

Image — Test 02 forward ping from App-A

Figure 17. Test 02, App-A, after routes: ping -c 4 10.20.1.10 gets 4 of 4 replies, 0% packet loss.

Image — Test 03 reverse ping from App-B

Figure 18. Test 03, App-B (ip-10-20-1-10), after routes: ping -c 4 10.10.2.10 gets 4 of 4 replies, 0% packet loss. hostname and hostname -I shown.

Image — Test 04 SSH to App-B through the bastion

Figure 19. Test 04, laptop: ssh ewb-app-b opens a shell on App-B (10.20.1.10) through the bastion using ProxyJump.

Image — Test 05 App-A public IPv4 empty

Figure 20. Test 05, App-A instance details: Public IPv4 address empty, private IP 10.10.2.10.

Image — Test 05 App-B public IPv4 empty

Figure 21. Test 05, App-B instance details: Public IPv4 address empty, private IP 10.20.1.10.

Controlled failure and recovery

Removing the ICMP Echo rule stopped the ping, and restoring it brought the ping back. I removed only the ICMP Echo request rule (source 10.10.2.10/32) from ewb-sg-app-b and kept the SSH rule. The ping from App-A to 10.20.1.10 then received no replies (100% packet loss). After I restored the exact same rule, the ping succeeded again with 4 packets received and 0% packet loss.

This shows that an Active peering connection and correct routes are not enough on their own. The security group on the destination must also allow the traffic.

Image — ewb-sg-app-b inbound rules with ICMP removed

Figure 22. ewb-sg-app-b inbound rules with the ICMP Echo rule removed: only SSH, TCP 22 from 10.10.1.10/32 remains.

Image — App-A ping failing after the rule was removed

Figure 23. App-A: the earlier ping gets 4 of 4 replies; after the ICMP rule is removed, ping -c 4 10.20.1.10 gets 0 replies, 100% packet loss.

Image — ewb-sg-app-b inbound rules with ICMP restored

Figure 24. ewb-sg-app-b inbound rules with the ICMP rule restored for 10.10.2.10/32 alongside the SSH rule.

Image — App-A ping succeeding again

Figure 25. App-A after recovery: ping -c 4 10.20.1.10 gets 4 of 4 replies, 0% packet loss.

Forward path and return path

A ping from App-A to App-B needs a route and a security rule in both directions.

- **Forward path:** App-A (10.10.2.10) sends a ping to 10.20.1.10. The route 10.20.0.0/16 in ewb-rt-a-private sends it to the peering connection, which delivers it to VPC B. ewb-sg-app-b allows the ICMP Echo request from 10.10.2.10/32.
- **Return path:** App-B replies to 10.10.2.10. The route 10.10.0.0/16 in ewb-rt-b-private sends the reply back through the peering connection. Security groups are stateful, so the reply is allowed automatically.
- **Bastion to App-B:** the bastion opens its own SSH connection to App-B, so ewb-rt-a-public also needs the 10.20.0.0/16 peering route, and ewb-sg-app-b allows SSH from the bastion's private IP (10.10.1.10/32).

Cleanup evidence

All resources created for this project were deleted in this order:

1. Instances (ewb-bastion, ewb-app-a, ewb-app-b)
2. Peering routes and the peering connection (ewb-a-b-peer)
3. Security groups
4. Subnets
5. Route tables
6. Internet gateway (ewb-igw-a)
7. VPCs (ewb-vpc-a and ewb-vpc-b)
8. Key pair (ewb-lab) and the local key file

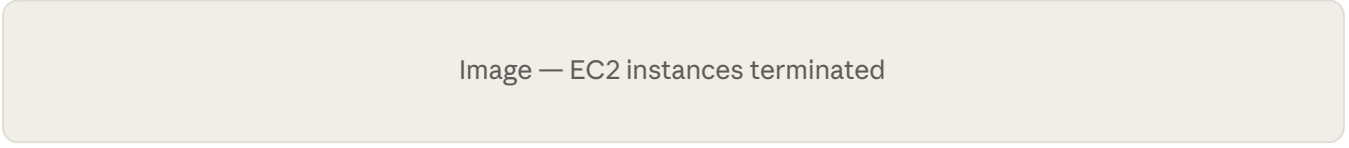


Figure 26. EC2 instances page: termination of ewb-bastion, ewb-app-a and ewb-app-b initiated, all three showing Terminated.

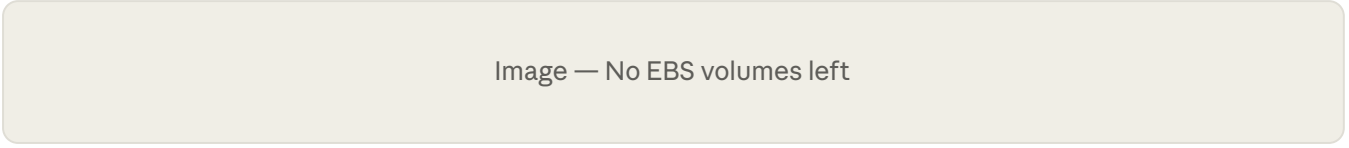


Figure 27. EC2 Volumes page: no volumes in the Region, so no lab volumes were left behind.

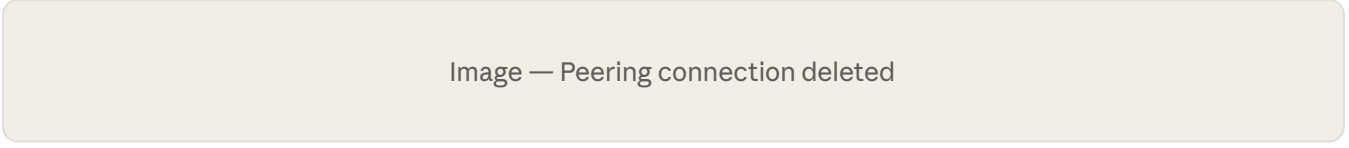


Figure 28. Peering connections page: pcx-018895e855a398629 (ewb-a-b-peer) deleted, status Deleted.

Image — No VPCs left

Figure 29. Your VPCs page: no VPCs found in the Region, so ewb-vpc-a and ewb-vpc-b are deleted.

Image — No subnets left

Figure 30. Subnets page: no subnets found.

Image — No route tables left

Figure 31. Route tables page: no route tables found.

Image — No internet gateways left

Figure 32. Internet gateways page: no internet gateways found, so ewb-igw-a is deleted.

Image — Tag Editor search with no results

Figure 33. Tag Editor in ap-south-1, all supported resource types, tags Project = EWB-VPC-Peering and Student = Jiya: no resources found.

What I learned

VPC peering is only the link between two VPCs, and traffic uses it only when both sides have the right routes. I needed a route in each private route table (and in the public one for the bastion) pointing at the peering connection, because replies need a path back as well as the request.

Security groups still decide what is allowed, and the controlled failure showed this: removing the ICMP Echo rule stopped the ping even though the peering and routes were fine.

Using a bastion with ProxyJump let me reach both private servers without giving them public IPs or copying the key onto any instance. App-B sees the bastion's private IP as the SSH source, so that is the address its security group rule must allow.

I also learned that peering is non-transitive and gives VPC B no internet access through VPC A, and that SSH is strict about file permissions on keys and config files.